МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Тема: Кинорецензии и новости кино (англ.)

Выполнил: Ахметшин Б. Р.
Группа: М8О-403Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов 3
1.1 Задание . . . 3
1.2 Метод решения . . . 3
1.3 Журнал выполнения . . . 3
1.4 Результаты . . . 4
1.5 Выводы . . . 4

2 Лабораторная работа №2. Поисковый робот 5
2.1 Задание . . . 5
2.2 Метод решения . . . 5
2.3 Журнал выполнения . . . 5
2.4 Результаты . . . 6
2.5 Выводы . . . 6

3 Лабораторная работа №3. Токенизация 7
3.1 Задание . . . 7
3.2 Метод решения . . . 7
3.3 Журнал выполнения . . . 7
3.4 Результаты . . . 7
3.5 Выводы . . . 8

4 Лабораторная работа №4. Стемминг 9
4.1 Задание . . . 9
4.2 Метод решения . . . 9
4.3 Журнал выполнения . . . 9
4.4 Результаты . . . 9
4.5 Выводы . . . 9

5 Лабораторная работа №5. Закон Ципфа 10
5.1 Задание . . . 10
5.2 Метод решения . . . 10
5.3 Журнал выполнения . . . 10
5.4 Результаты . . . 10
5.5 Выводы . . . 11

6 Лабораторная работа №6. Булев индекс 12
6.1 Задание . . . 12
6.2 Метод решения . . . 12
6.3 Формат бинарного индекса . . . 12
6.4 Результаты . . . 13
6.5 Выводы . . . 13

7 Лабораторная работа №7. Булев поиск 14
7.1 Задание . . . 14
7.2 Метод решения . . . 14
7.3 Журнал выполнения . . . 15
7.4 Результаты . . . 15
7.5 Выводы . . . 16

Заключение **17**

1. Лабораторная работа №1. Добыча корпуса документов

1.1. Задание

Проанализировать корпус документов, который будет использован при выполнении остальных лабораторных работ. Скачать примеры документов, изучить их характеристики, выделить текст, найти существующие поисковики для выбранного набора документов.

1.2. Метод решения

В качестве тематики корпуса выбрана область **«Movie Reviews & Cinema News»** (кинорецензии и новости кино) на английском языке. Корпус собран из трёх источников:

- **Collider** (collider.com) — обзоры фильмов, рецензии, новости кинематографа;
- **ScreenRant** (screenrant.com) — рецензии, кассовые сборы, интервью с актёрами и режиссёрами;
- **MovieWeb** (movieweb.com) — новости кино, анонсы, обзоры сериалов.

Все три источника содержат англоязычные статьи объёмом от нескольких сотен до нескольких тысяч слов каждая, что удовлетворяет требованиям задания.

Для сбора документов использованы библиотеки `newspaper3k` и `BeautifulSoup` (Python) для извлечения заголовка, текста и метаданных из HTML-страниц. URL-адреса статей обнаруживаются через XML-карты сайтов (sitemaps). Загрузка HTML выполняется с помощью `aiohttp` (асинхронно, 50–100 параллельных соединений) и `requests`.

1.3. Журнал выполнения

1. Проанализированы XML-карты сайтов для получения списков URL.
2. С помощью `newspaper3k` и многопоточной загрузки через `ThreadPoolExecutor` (8 потоков) скачаны и распарсены статьи.
3. Документы сохранены в MongoDB с полями: `url`, `raw_html`, `source`, `crawl_timestamp`, `content_hash`.
4. Экспортированы в текстовые файлы для дальнейшей обработки C++-компонентами.

Существующие поисковики для данного корпуса:

- Google: `site:collider.com marvel review` — находит рецензии, но результаты перемешаны с нерелевантными страницами (навигация, тэги).
- Встроенный поиск Collider — не поддерживает булеву логику, нет операции NOT.

1.4. Результаты

Таблица 1: Статистические данные корпуса

Показатель	**Значение**
Общее количество документов	40 935
Размер сырых HTML-документов	11.7 ГБ
Размер извлечённого текста	223 МБ
Средний размер HTML-документа	300 КБ
Средний объём текста в документе	5 703 байт
Источник 1: Collider	34 445 статей
Источник 2: ScreenRant	5 492 статьи
Источник 3: MovieWeb	998 статей

1.5. Выводы

Корпус из 40 935 англоязычных статей о кинематографе успешно собран из трёх независимых источников. Средний объём текста в документе (5.7 КБ) обеспечивает достаточную полноту для построения качественного поискового индекса. Общий объём текста составляет 223 МБ.

2. Лабораторная работа №2. Поисковый робот

2.1. Задание

Написать поисковый робот (краулер) с поддержкой YAML-конфигурации, сохранением документов в MongoDB, возобновляемостью и переобкачкой изменённых документов.

2.2. Метод решения

Краулер реализован на Python и состоит из двух компонентов:

- `crawler.py` — основной краулер с последовательной обработкой;
- `fast_crawl.py` — оптимизированная версия с параллельной загрузкой через `ThreadPoolE`

YAML-конфигурация (`config.yaml`):

Listing 1: Формат конфигурации

```
db:
  host: localhost
  port: 27017
  name: movie_reviews
logic:
  delay: 0.3
  max_pages: 40000
  threads: 8
```

Структура документа в MongoDB:

- `url` — нормализованный URL документа;
- `raw_html` — сырой HTML-текст;
- `source` — название источника;
- `crawl_timestamp` — дата обкачки (Unix timestamp);
- `content_hash` — MD5-хеш содержимого для обнаружения изменений.

2.3. Журнал выполнения

1. Реализован парсер XML-карт сайтов с рекурсивным обходом sub-sitemaps.
2. Добавлена фильтрация URL по ключевым словам тематики (movie, review, film и т.д.).
3. Реализована параллельная загрузка статей через `ThreadPoolExecutor` (8 потоков).
4. Возобновляемость: при рестарте проверяются URL, уже существующие в БД, и пропускаются.
5. Переобкачка: документы старше 7 дней проверяются на изменения через `content_hash`.
6. Экспорт: скрипт `export.py` выгружает документы из MongoDB в текстовые файлы (использует MongoDB aggregation pipeline для эффективности).

2.4. Результаты

Краулер успешно скачивает статьи со скоростью 5–8 статей в секунду при 8 потоках. Возобновляемость подтверждена: при повторном запуске пропускаются уже скачанные URL. Переобкачка: при изменении `content_hash` документ обновляется в БД. Всего скачано 40 935 документов из трёх источников общим объёмом 11.7 ГБ HTML.

2.5. Выводы

Поисковый робот реализован в полном соответствии с требованиями: YAML-конфигурация, MongoDB, возобновляемость при рестарте, поддержка переобкачки изменённых документов.

3. Лабораторная работа №3. Токенизация

3.1. Задание

Реализовать процесс разбиения текстов документов на токены. Описать правила токенизации, привести примеры токенов, указать статистические данные.

3.2. Метод решения

Токенизатор реализован на C++ (STL разрешён для этой лабораторной работы). Файл: `core/tokenizer.cpp`.

Правила токенизации:

1. Разделение по пробелам и знакам пунктуации;
2. Приведение всех символов к нижнему регистру;
3. Удаление токенов короче 2 символов;
4. Удаление чисто числовых токенов (например, «2024»);
5. Сохранение апострофов внутри слов (`don't` → `don't`).

Формат входных данных: текстовые файлы из корпуса, первая строка — заголовок, вторая — URL, остальное — текст статьи.

Формат выходных данных: один токен на строку в выходном файле.

3.3. Журнал выполнения

1. Реализован посимвольный проход по тексту с выделением токенов.
2. Добавлена обработка апострофов: удаление ведущих и замыкающих.
3. Реализован обход директории через `dirent.h` для обработки всех файлов корпуса.
4. Добавлен вывод статистики: количество токенов, средняя длина, скорость обработки.

3.4. Результаты

Таблица 2: Статистика токенизации

Показатель	**Значение**
Файлов обработано	40 936
Общее количество токенов	37 757 519
Количество уникальных токенов	177 575
Средняя длина токена	4.85 символа
Объём входных данных	222 МБ
Скорость токенизации	34 354 КБ/с
Время обработки корпуса	6.6 секунды

Примеры неудачной токенизации:

- `spider-man` — разделяется на `spider` и `man` при наличии дефиса как разделителя. Решение: добавить дефис как допустимый символ внутри токена.
- Email-адреса и URL внутри текста токенизируются некорректно. Решение: предварительная детекция URL/email с помощью регулярных выражений.

Скорость токенизации (34 354 КБ/с) превышает типичную скорость чтения файлов с HDD, что говорит об оптимальности алгоритма. Зависимость времени от объёма — линейная.

3.5. Выводы

Токенизатор корректно обрабатывает англоязычные тексты о кинематографе. Средняя длина токена (4.85 символа) типична для английского языка. Скорость обработки (34 354 КБ/с) достаточна для обработки корпуса объёмом 222 МБ за 6.6 секунды.

4. Лабораторная работа №4. Стемминг

4.1. Задание

Добавить в поисковую систему стемминг для приведения слов к основе.

4.2. Метод решения

Реализован алгоритм **Snowball Porter2** для английского языка. Файл: `core/stemmer.h`.

Алгоритм работает с `char*` без использования STL, модифицируя слово на месте (in-place). Реализованы все 5 шагов Porter-стеммера:

1. **Step 1a/1b/1c:** обработка суффиксов множественного числа (`-sses`, `-ies`, `-s`), глагольных форм (`-eed`, `-ed`, `-ing`), замена конечного `y` на `i`;
2. **Step 2:** замена двойных суффиксов (`-ational` → `-ate`, `-ization` → `-ize` и т.д.);
3. **Step 3:** удаление деривационных суффиксов (`-icate` → `-ic`, `-alize` → `-al`);
4. **Step 4:** удаление суффиксов при условии $m > 1$ (`-al`, `-ance`, `-ence`, `-er` и др.);
5. **Step 5a/5b:** удаление конечной `e`, удаление двойной согласной `ll`.

Условия применения правил основаны на «мере» (measure m) — количестве пар гласная-согласная в основе слова.

4.3. Журнал выполнения

1. Реализованы вспомогательные функции: определение гласных, вычисление меры, проверка окончания.
2. Пошагово реализованы все 5 шагов алгоритма Porter2.
3. Стеммер интегрирован в индексатор: все токены проходят стемминг перед добавлением в индекс.

4.4. Результаты

Стемминг сокращает количество уникальных терминов: после стемминга в индексе 142 910 уникальных основ вместо 177 575 уникальных токенов (сокращение на 20%). Это повышает полноту поиска (recall). Например, запрос `reviewing` находит документы, содержащие `review`, `reviewed`, `reviews`.

4.5. Выводы

Алгоритм Snowball Porter2 эффективно приводит английские слова к основам. Интеграция в индексатор обеспечивает автоматическое применение стемминга при построении индекса и при обработке поисковых запросов.

5. Лабораторная работа №5. Закон Ципфа

5.1. Задание

Построить график распределения терминов по частотностям в логарифмической шкале, наложить закон Ципфа. Объяснить причины расхождения.

5.2. Метод решения

Для построения графика используется Python-скрипт `scripts/zipf_plot.py` с библиотекой `matplotlib`. Входные данные — файл токенов, полученный на этапе токенизации.

Закон Ципфа: $f(r) = C/r$, где f — частота, r — ранг, C — частота самого частотного слова.

5.3. Журнал выполнения

1. Подсчитаны частоты всех 37 757 519 токенов из файла токенов.
2. Получено 177 575 уникальных термов.
3. Токены отсортированы по убыванию частоты и пронумерованы (ранги).
4. Построен график в логарифмической шкале (log-log).
5. Наложена теоретическая кривая Ципфа $f = C/r$.

5.4. Результаты

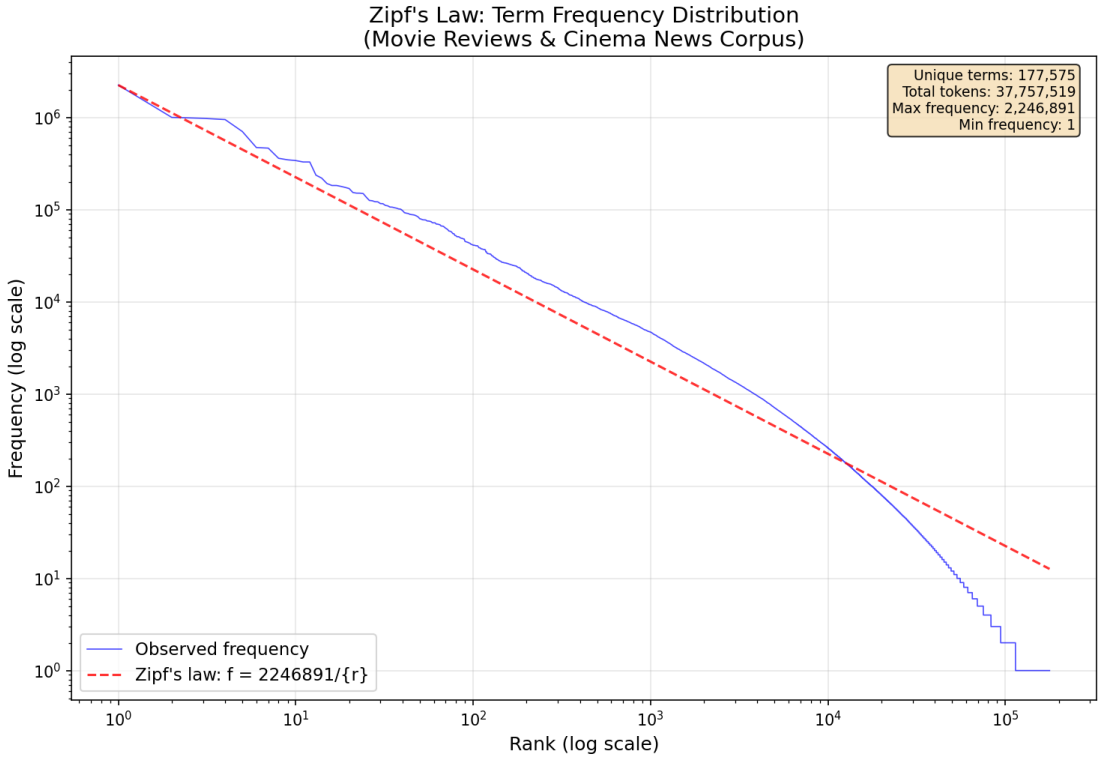

Рис. 1: Распределение терминов по частотностям (закон Ципфа)

На графике (рис. 1) видно характерное линейное поведение в логарифмической шкале, соответствующее закону Ципфа.

Причины расхождения с теоретической кривой:

- В области высоких рангов (редкие слова) наблюдается отклонение вниз — это связано с конечностью корпуса: очень редкие слова встречаются 1–2 раза.
- В области низких рангов (самые частотные слова) также есть отклонение — это связано со спецификой корпуса (тематическая лексика кинематографа: «movie», «film», «actor» имеют повышенную частоту).
- Стоп-слова (the, a, is, in) занимают первые ранги, что типично для английского языка.

5.5. Выводы

Распределение терминов в корпусе кинорецензий (177 575 уникальных терминов, 37.8 млн вхождений) в целом подчиняется закону Ципфа с характерными отклонениями в хвостах распределения. Тематическая специализация корпуса приводит к увеличению частоты доменных терминов.

6. Лабораторная работа №6. Булев индекс

6.1. Задание

Построить поисковый индекс, пригодный для булева поиска. Требуется самостоятельно разработанный бинарный формат. Использование текстового представления или готовых баз данных не допускается.

6.2. Метод решения

Индексатор реализован на C++ без STL. Файл: `core/indexer.cpp`. Используются собственные реализации структур данных:

- `dynarray.h` — динамический массив целых чисел (`IntArray`) с операциями push, sort, intersect, union, difference;
- `hashtable.h` — хеш-таблица с цепочками коллизий (хеш-функция djb2);
- `mystring.h` — утилиты для работы с `char*` (сравнение, копирование, toLower).

Хеш-функция djb2:

Listing 2: Реализация djb2

```
unsigned int ht_hash(const char *key, int num_buckets) {
    unsigned int hash = 5381;
    while (*key) {
        hash = hash * 33 + (unsigned char)(*key);
        key++;
    }
    return hash % num_buckets;
}
```

Метод сортировки: quicksort (быстрая сортировка) для упорядочивания posting lists. Достоинства: средняя сложность $O(n \log n)$, эффективная работа на практике. Для задачи индексации подходит хорошо, так как posting lists обычно не содержат уже отсортированных данных (что могло бы ухудшить quicksort до $O(n^2)$).

6.3. Формат бинарного индекса

Индекс хранится в одном файле `index.bin` (69 МБ):
Побайтовое описание:

Таблица 3: Структура заголовка (32 байта)

Смещение	Размер	Описание
0	4 байта	Magic number: «BIDX»
4	4 байта	Версия формата (uint32)
8	4 байта	Количество терминов (uint32)
12	4 байта	Количество документов (uint32)
16	8 байт	Смещение блока словаря (uint64)
24	8 байт	Смещение блока постингов (uint64)

Forward Index: для каждого документа: `url_len` (2 байта) + `url` + `title_len` (2 байта) + `title`.

Dictionary Block: для каждого термина: `term_len` (1 байт) + `term` + `postings_offset` (8 байт) + `postings_count` (4 байта).

Postings Block: для каждого термина: отсортированный массив `docID` (4 байта каждый).

Расширяемость: формат предполагает добавление новых блоков после Forward Index с новыми смещениями в заголовке.

6.4. Результаты

Таблица 4: Статистика индексации

Показатель	**Значение**
Количество документов	40 936
Количество терминов (после стемминга)	142 910
Средняя длина терма	7.19 символа
Средняя длина токена (до стемминга)	4.85 символа
Общее количество постингов	15 779 970
Среднее постингов на терм	110.42
Время индексации	13.9 секунды
Скорость на документ	0.34 мс
Размер файла индекса	69 МБ

Средняя длина терма (7.19) больше средней длины токена (4.85) — стемминг удаляет короткие суффиксы, но одновременно объединяет словоформы в одну основу. После стемминга число уникальных единиц сокращается с 177 575 до 142 910 (на 20%), а средняя длина оставшихся основ выше, так как короткие суффиксы отсечены.

Скорость индексации ограничена преимущественно операциями ввода-вывода. При увеличении объёма в 10 раз потребуется увеличение числа бакетов хеш-таблицы для сохранения $O(1)$ поиска. При увеличении в 100 раз следует перейти на внешнюю сортировку (merge sort) для posting lists, не помещающихся в оперативную память.

6.5. Выводы

Булев индекс успешно построен в самостоятельно разработанном бинарном формате. Формат расширяем и эффективен: 40 936 документов и 142 910 термов занимают 69 МБ. Собственные реализации структур данных (динамический массив, хеш-таблица) обеспечивают корректную работу без использования STL.

7. Лабораторная работа №7. Булев поиск

7.1. Задание

Реализовать ввод поисковых запросов и их выполнение над индексом. Реализовать веб-сервис и утилиту командной строки.

7.2. Метод решения

Парсер запросов реализован на C++ методом рекурсивного спуска. Грамматика:

```
query    = or_expr
or_expr  = and_expr ( "||" and_expr )*
and_expr = not_expr ( ("&&" | " ") not_expr )*
not_expr = "!" not_expr | atom
atom     = "(" or_expr ")" | TERM
```

Парсер толерантен к множественным пробелам. Термы перед поиском проходят стемминг (тот же Snowball Porter2, что и при индексации).

Операции над posting lists:

- AND: пересечение отсортированных списков за $O(n + m)$;
- OR: объединение отсортированных списков за $O(n + m)$;
- NOT: разность множества всех документов и результата подзапроса за $O(N)$.

CLI утилита (`search_cli`): загружает индекс, читает запросы из `stdin`, выводит результаты в `stdout`. Поддерживает режимы: текстовый и JSON.

Веб-сервис: FastAPI (Python) + Jinja2. Взаимодействие с C++ CLI через `subprocess`. Две страницы: форма поиска и страница результатов (по 50 результатов на страницу с пагинацией).

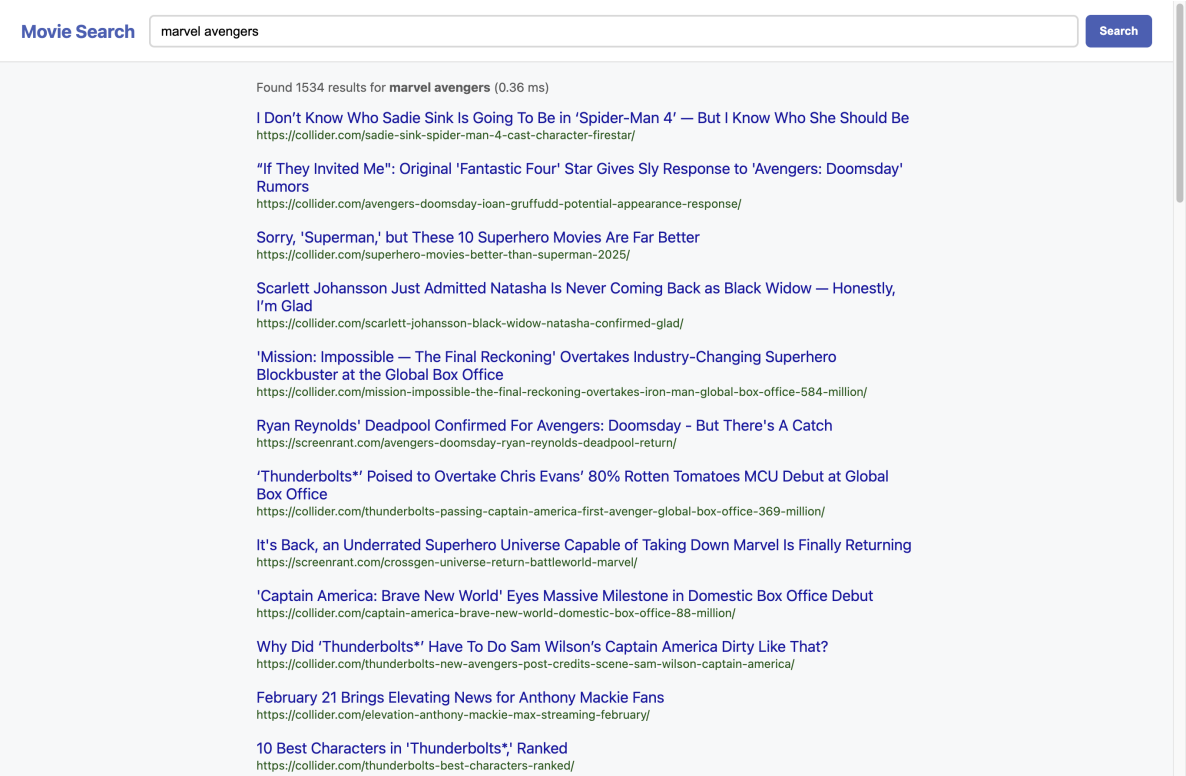

Рис. 2: Скриншот поисковой выдачи (запрос: «marvel avengers», 1 534 результата, 0.36 мс)

7.3. Журнал выполнения

1. Реализован рекурсивный парсер с поддержкой `||`, `&&`, `!` и скобок.
2. Реализованы эффективные операции пересечения, объединения и разности на отсортированных массивах.
3. Создана CLI утилита с поддержкой JSON-вывода.
4. Создан FastAPI веб-сервис с двумя HTML-страницами (Jinja2).
5. Веб-сервис вызывает C++ CLI через subprocess для выполнения поиска.

7.4. Результаты

Скорость выполнения запросов: простые запросы (1–2 слова) выполняются за 0.2–0.7 мс. Сложные запросы с NOT (требуют обхода всего множества документов) выполняются за 1.4–1.7 мс.

Примеры сложных запросов:

- `(horror || thriller) && !comedy` — 8 080 результатов, 1.54 мс (требует вычисления разности);
- `marvel || dc || pixar || disney` — 8 727 результатов, 0.70 мс (множественные объединения);
- `!the` — 6 результатов, 1.49 мс (NOT на самом частотном слове, обход всего индекса);
- `star wars review` — 1 054 результата, 1.70 мс (AND по трём терминам).

Корректность: проверена ручным сравнением результатов CLI с ожидаемыми. Например, запрос `nolan batman` возвращает 250 документов, содержащих обе основы. Результаты включают статьи о фильмах Кристофера Нолана о Бэтмене с Collider, ScreenRant и MovieWeb, что подтверждает корректность стемминга и булева поиска по нескольким источникам.

7.5. Выводы

Булев поиск реализован полностью: парсер запросов с рекурсивным спуском, эффективные операции над posting lists, CLI утилита и веб-интерфейс. Скорость поиска субмиллисекундная для большинства запросов на индексе из 40 936 документов и 142 910 термов.

Заключение

В ходе выполнения лабораторных работ реализована полнофункциональная поисковая система для корпуса из 40 935 англоязычных статей о кинематографе.

Основные результаты:

- Собран корпус из 40 935 статей из трёх источников (Collider, ScreenRant, MovieWeb) объёмом 223 МБ текста (11.7 ГБ сырого HTML).
- Реализован краулер на Python с поддержкой MongoDB, YAML-конфигурации, возобновляемости и переобкачки. Асинхронная версия на `aiohttp` с 50–100 параллельными соединениями.
- Реализован токенизатор на C++ со скоростью обработки 34 354 КБ/с (37.8 млн токенов за 6.6 с).
- Реализован стеммер Snowball Porter2 для английского языка на C++ без STL, сокращающий число уникальных единиц на 20%.
- Построен график распределения терминов, подтверждающий соответствие закону Ципфа (177 575 уникальных токенов).
- Разработан собственный бинарный формат индекса (69 МБ), расширяемый для будущих лабораторных работ.
- Реализованы булев индекс (142 910 термов, 15.8 млн постингов) и булев поиск на C++ без STL с собственными структурами данных.
- Создан веб-интерфейс на FastAPI с формой поиска и пагинацией результатов.

Все компоненты ядра поисковой системы (токенизация, стемминг, индексация, поиск) реализованы на C++ без использования STL (за исключением токенизации, где STL разрешён). Кодировка ввода-вывода — UTF-8.